Blockchains and Distributed Ledgers
Lecture 10 – Part 1

Damiano Abram

19 November 2025

1 Actively Secure MPC

In Lecture 9, we saw how to build multiparty computation protocols (MPC) with passive security, which means that privacy and correctness of the computation were guaranteed as long as the set of (at most $t-1$) colluding parties followed the protocol. Essentially, the threat was that, by sharing all the information they had about the protocol execution, they could recover, at least partially, the inputs of the remaining participants.

In this lecture, we will investigate MPC protocols with active security, meaning that privacy and correctness will be guaranteed even if the (at most $t-1$) colluding parties misbehave. Actively secure MPC is a quite complex world, and investigating all its aspects would be too much for this course. We will therefore focus on two important and apparently simple problems: *secure coin tossing* and *fair swap*. Both problems are extremely easy to state, but solving them highlights one of the main challenges in actively secure MPC.

1.1 Coin Tossing

Our setting is extremely simple. Imagine that we have N parties, and the goal is for them to agree on a random bit (often we call this a *random coin*, given that we can view the problem as that of digitally tossing a coin, 0 means head, 1 means tail). In other words, at the end of the protocol, they must all output the same bit b.

This is a input-less computation, so we do not have to worry about privacy (there is nothing we need to keep secret), however, the challenge is to guarantee correctness: doesn't matter how the $t-1$ colluding parties behave, the output should be uniformly random. For example, an *insecure* solution would be to delegate the generation of the random coin to one of the participants (which would then broadcast it to everybody else). If the delegated party were to be corrupted, they could simply choose the output to be what they liked the most.

1.2 Coin Tossing with Abort

Let's consider the (apparently) simplest setting: suppose there are only two parties, Alice and Bob, and we want to achieve security as long as at most one of them misbehaves. If we are willing to take compromises, there is a relatively easy solution for Alice and Bob to securely toss a coin.

1. (Round 1) Alice samples a random bit $x \xleftarrow{\$} \{0,1\}$ and sends a (binding and hiding) commitment of x to Bob. Let c be the commitment and let r be its opening.

2. (Round 2) Once Bob received c, he samples another random bit $y \xleftarrow{\$} \{0,1\}$ and sends it directly to Alice.

3. (Round 3) Once Alice receives y, she reveals r and x to Bob. Then, she outputs the XOR $x \oplus y$.

4. Bob checks that (x, r) is a valid opening to c. If so, he outputs $x \oplus y$.

Let's analyse why this protocol is secure. First, of all, it is easy to see that, when Alice and Bob are both honest, they output the same value $x \oplus y$. Notice that this corresponds to the XOR of two random bits, which is also uniformly random. However, this is not the end of the analysis, as we need to make sure that the output is random even if one of the two participants misbehaves.

Case 1: Alice is honest, Bob is malicious. This case is simple. By the hiding properties of the commitment scheme, c hides the random bit x chosen by Alice. Bob can generate y in arbitrary ways (he is malicious), however, since x is hidden, y will be independent of x (Alice reveals x, only after receiving y). We conclude that the output $x \oplus y$ will be the XOR between a value y and a *uniformly random* and *independent* bit x. This is also uniformly random.

Case 2: Alice is malicious, Bob is honest. Since Alice misbehaves, she may not choose x at random[1] This is, however, not an issue, as, by the binding properties, Alice knows at most one way to open c in round 3 [2] Crucially, the only known opening (x, r) is fixed before she receives y from Bob. In conclusion, since Bob is honest, y is random and *independent* of x. The XOR between any bit x and any *uniformly random* and *independent* bit y is also random.

What if Alice aborts? The analysis in case 2 hides a little issue: what if Alice refuses to send a valid opening of c in round 3? In this way, she would prevent Bob from computing the output. We call this an *abort*. In general, aborts are not necessarily a problem as long as Alice cannot decide to abort based on the outcome of the protocol. However, this is not the case in our protocol. Indeed, after receiving y, Alice can immediately compute the output of the protocol, and so, she can decide to abort the execution if she does not like the outcome. We call this a *selective abort*. The outcome of the protocol is not truly random: it is Alice's favourite output with probability $1/2$ and abort with probability $1/2$. This can be very problematic for Bob, as it can reduce the probability of an infavourable outcome for Alice to 0. Due to this issue, we say that our protocol is a coin tossing protocol *with abort*: the output is guaranteed to be random as long as the misbehaving parties commit to terminate the protocol execution. Not a very good assumption in the real world.

1.3 Fair Coin Tossing

Can we build a coin tossing protocol that is secure even if parties decide to abort? The issue in the previous construction was that the protocol was not *fair*: one party got to learn the output before the other, and, by refusing to further participate in the execution, they could prevent the other from receiving the output. Unfortunately, fair coin tossing is impossible when we have just two parties and we want security as long as at most one of them misbehaves. More generally, the problem has no solution even if we have N parties and we want to withstand any set of $t - 1$ misbehaving parties, where $t \geq N/2 + 1$. We can however solve the problem when the majority of participants follows the protocol.

Verifiable secret-sharing. To build our protocol, we need a new tool: *verifiable secret-sharing* (VSS). Like any t-out-of-n secret-sharing, this is a scheme that allows us to recover the secret x only if we pool together at least t chunks, while, with any smaller number of chunks, x remains completely secret. There are however new features that provide security guarantees against misbehaving parties:

- If who generated the shares was honest and at least t honest parties participate to the reconstruction, the recovered secret is exactly the original one that was secret-shared. This means that the malicious participants cannot prevent the reconstruction by infiltrating the pool and injecting bogus shares.

[1] Even worse, she might even generate c in arbitrary ways. Maybe c does not even represent a valid commitment!
[2] If c is not a valid commitment, most likely Alice knows no way to open it.

- Even if who generated the shares was malicious, there still exists a value x such that, if at least t honest parties participate to the reconstruction, the recovered secret is x. This holds even if the malicious participants try to prevent the reconstruction by injecting bogus shares.

A fair coin tossing protocol. Suppose we have N parties $P_1, \ldots, P_N$. We want to design a fair coin tossing protocol that is secure even when at most $t < N/2 + 1$ of them misbehave (and potentially collude). We can do it as follows:

1. (Round 1) Each party P_i samples a uniformly random bit $x_i \xleftarrow{\$} \{0,1\}$ and distributes a t-out-of-N verifiable secret-sharing of x_i.
2. (Round 2) When everyone has received a share from everyone else, the parties pool together their shares to reconstruct every x_i. They output $x_1 \oplus \cdots \oplus x_N$.

Why is this protocol secure? Let C denote the set of indexes of corrupted parties, let H denote the indexes of the honest parties. Observe that $|H| \geq N - t + 1 > N/2 > t - 1$, so $|H| \geq t$.

By the second property of VSS, the secret-sharings produced by malicious participants still binds them to some values. This means that, since there are at least t honest parties involved, the reconstruction procedure will always give the same result $(x_j)_{j \in C}$, no matter how the malicious parties try to disrupt it.

The values $(x_j)_{j \in C}$ will also be independent of the random bits $(x_i)_{i \in H}$ chosen by the honest parties. This is because we used a t-out-of-N secret-sharing and there are at most $t-1$ misbehaving parties. In other words, the set of colluding parties needs to distribute their shares when the values $(x_i)_{i \in H}$ are still secret.

Finally, by the first property of VSS, the malicious parties cannot even prevent the reconstruction of $(x_i)_{i \in H}$. To conclude, the final output will be obtained by XORing the (not necessarily random) bits $(x_j)_{j \in C}$ with the *random* and *independent* bits $(x_i)_{i \in H}$. The result will therefore be random.

1.3.1 How do we build VSS?

Verifiable secret-sharing is a tool that has been studied by cryptographers for a long time. There are therefore a lot of constructions, each of them with their own advantages and disadvantages. In these notes, we provide a rather naïve solution using the tools we have explored in the previous lectures: (standard) t-out-of-N secret-sharing, public-key encryption and non-interactive zero-knowledge arguments.

Suppose that we have access to a public-key infrastructure (PKI), meaning that each party P_i holds a key pair $(\mathsf{pk}_i, \mathsf{sk}_i)$ for a public-key encryption scheme, where pk_i is known to everybody and sk_i is known only to P_i. To generate a verifiable secret sharing of some value x, perform the following operations:

1. Derive a t-out-of-N secret-sharing of x (this is standard secret-sharing like the one we discussed in Lecture 9 for passively secure MPC). Let $x_1, \ldots, x_N$ be the shares.
2. For every i, derive a (hiding and binding) commitment c_i to x_i. Let r_i be the opening information.
3. For every i, derive an encryption ct_i of the pair (x_i, r_i) under pk_i.
4. Compute a non-interactive zero-knowledge argument π proving that
   - Each ct_i is an encryption of an opening (x_i, r_i) to c_i under the public key pk_i.
   - The values $x_1, \ldots, x_N$ are a valid t-out-of-N secret-sharing of a secret value x.

   Formally, the statement is the tuple $(\mathsf{pk}_i, c_i, \mathsf{ct}_i)_{i \in [N]}$, while the witness (which remains secret) consists of $x, (x_i, r_i)_{i \in [N]}$ and the randomness used by various procedures (secret-sharing, commitment and encryption).
5. The verifiable secret sharing of x consists of the *public* information $(\mathsf{ct}_i, c_i)_{i \in [N]}$ and π. Each party P_i obtains its share (x_i, r_i) by decrypting ct_i using their secret key.

To reconstruct the secret, the parties perform the following operations

1. Each party P_i broadcasts their share (x_i, r_i)
2. The parties check the argument π. If this does not verify, they output 0.
3. For every i, the parties check that (x_i, r_i) is a valid opening to c_i. If not, they ignore P_i's contribution.
4. The parties take all the shares that have not been discarded and, if there are at least t of them, they reconstruct the secret x from the corresponding elements x_i. Otherwise, they output $\mathsf{failure}$.

Why is this VSS scheme private? The public information $(\mathsf{ct}_i, c_i)_{i\in[N]}$ and π leak no information about x to any subset of at most $t-1$ colluding parties. Indeed, using their secret keys, the malicious parties could only recover at most $t-1$ shares $(x_i)_{i\in C}$. To obtain more of them, they would need to

1. either break the public-key encryption scheme and decrypt any of the ciphertexts $(\mathsf{ct}_i)_{i\in H}$;
2. or break the hiding properties of the commitments and recover the value hidden in any of the $(c_i)_{i\in H}$;
3. or break the zero-knowledge property of π and recover the witness from it.

Why can't the malicious parties disrupt the reconstruction? Suppose that the verifiable secret-sharing was generated honestly. Then, the shares (x_i, r_i) of the honest parties are a valid opening of c_i, so they are never discarded. On the other hand, for any $j \in C$, the malicious participants cannot open c_j to any value other than x_j. So, if they try to contribute to the reconstruction with any bogus pair (x'_j, r'_j) where $x_j \neq x'_j$, their share will be discarded. The output x will therefore be derived using the original shares for the standard t-out-of-N secret-sharing.

Why does a verifiable secret-sharing generated by misbehaving parties bind them to a value? If π does not verify, the verifiable secret-sharing can only reconstruct to 0. If instead π verifies, by the soundness property of the zero-knowledge argument, the verifiable secret-sharing coincides with an honestly generated secret sharing of some value x. By what we said in the previous paragraph, as long as t honest parties participate to the reconstruction, the recovered secret will always be x.

1.4 Fair Swap

It is time to introduce the second MPC protocol we are going to investigate in this quick trip into the world of active security: *fair swap*. The problem is simple. We have two parties, Alice and Bob, each holding some secret, and the goal is to exchange them. We assume that the parties have some easy way to verify that what they have received is indeed what they wanted. For instance, they could already hold a hash (or a commitment) of the other party's secret.

Since the inputs are going to be revealed anyway, we are not so much concerned with privacy. We are, however interested in achieving correctness even when one of the participants misbehaves. In particular, the exchange should be performed in a fair way: Bob should learn Alice's secret if and only if Alice learns Bob's secret. A typical setting where this can be useful is secret-sharing: we don't want the other participants to run away with our shares without disclosing theirs!

Let's analyse some broken attempts to build this primitive. Any solution in which one party reveals the secret before the other is doomed to fail, as whoever receives the secret first could just abort the protocol, never revealing their input to the other party. Also, sending the messages simultaneously would be a bad idea, as one party could just pretend to send their secret to trick the other one into revealing theirs.

So, how do we solve this problem? Well, once again the answer is that we cannot. Indeed, if we could, we could also build a 2-party fair coin tossing withstanding one misbehaving participant: Alice and Bob would first send to each other (hiding and binding) commitments to random bits x and y. Then, they would use the fair swap protocol to exchange the openings. The final output would be $x \oplus y$. The final output would be

random, and the fair swap protocol would prevent selective aborts. This gives a general recipe to build fair coin tossing from fair swap (looking ahead, we can use this idea to build a game-theoretic fair coin tossing from the game-theoretic fair swap we are going to discuss in a minute).

We solve our problems as before: we rely on external help. In the solution we are going to propose, this will be provided by the blockchain.

1.5 Game-Theoretic Fair Swap on a Blockchain

The idea is to set up a swap where Alice and Bob are disincentivised from deviating from the protocol via a system of rewards and penalties enforced by the chain. The main tool for this will be a particular type of transaction: *time-lock transactions*.

Time-lock transactions. A time-lock transaction is a particular transaction that additionally specifies the earliest time at which it can be added to the chain. For example, in a far future, grandpa Damiano could issue a time-lock transaction that sends all his cryptocurrency on the account of his granddaughter, setting the time parameter to the moment she turns 18. Even if Damiano signs the transaction today and broadcasts it to everybody, this cannot be validated and added to the chain as long as the granddaughter is underage.

Notice that this does not prevent Damiano from spending his money in other ways in the meantime, however, if he does so, the time-lock transaction will never be added to the chain because it would be in conflict with the more recent transactions issued by Damiano: the chain cannot have two transactions that spend the same money!

The protocol. Let us go back to our fair swap problem. Suppose that Alice's secret is x_0, while Bob's secret is x_1. Suppose also that Alice holds a hash y_1 of x_1, while Bob holds a hash y_0 of x_0. In this way, the parties can validate any value they receive against these hashes. If the check goes through, they are essentially sure that what they received is indeed what they wanted (otherwise, the other party managed to find a collision for the hash function).

The parties start by agreeing on an upper bound B on the monetary value of their secrets. Alice will prepare two transactions:

TX_A: This transaction moves $\$B$ from Alice's account to a smart contract. The money in the contract can be sent to Bob's account if he provides a pair (x_0, x_1) such that $y_0 = \mathsf{Hash}(x_0)$ and $y_1 = \mathsf{Hash}(x_1)$. The money can alternatively be redeemed by Alice by providing a transaction signed by both Alice and Bob having input TX_A.

TX'_A: This is a time-lock transaction that sends the money of TX_A back to Alice's account. The time parameter is set to a time t_A.

Similarly, Bob prepares two other transactions:

TX_B: This transaction moves $\$B$ from Bob's account to a smart contract. The money in the contract can be sent to Alice's account if she provides a value x_0 such that $y_0 = \mathsf{Hash}(x_0)$. Alternatively, Bob can redeem the money by providing a transaction signed by both Alice and Bob having input TX_B.

TX'_B: This is a time-lock transaction that sends the money of TX_B back to Bob's account. The time parameter is set to a time t_B.

The parties proceed as follow:

1. Alice sends the transaction TX'_A to Bob. Bob sends the transaction TX'_B to Alice.
2. Bob sends a signature on TX'_A back to Alice. Alice sends a signature on TX'_B back to Bob. If any of the signatures do not verify, the parties abort.

3. Alice signs TX_A and broadcasts it to the miners so that it can be added to the chain.
4. Once TX_A has been validated, Bob signs TX_B and broadcasts it to the miners so that it can be added to the chain.
5. Alice redeems the money in TX_B by revealing x_0 (x_0 becomes public because it is stored on the chain). The amount is sent to her account.
6. If Alice has spent the money in TX_B revealing x_0 in the process, Bob redeems the money in TX_A by revealing (x_0, x_1) (x_1 becomes public because it gets stored on the chain). The amount is sent to his account. If instead Alice has not claimed the money in TX_B by time t_B, Bob signs TX'_B and broadcasts it to the miners so that it can be added to the chain. In this way, Bob is reimbursed for $\$B$.
7. If Bob has not claimed the money in TX_A by time t_A, Alice signs TX'_A and broadcasts it to the miners so that it can be added to the chain. In this way, Alice is reimbursed for $\$B$.

Notice that if everyone follows the protocol, the parties exchange their secrets without losing or gaining any money. If instead one party aborts after the other party's secret is revealed, they lose $\$B$, while the deceived party gains $\$B$. Finally, if a party aborts before the other participant's input is revealed, nobody gains or loses anything. In this way, parties are disincentivised from misbehaving. Below, we discuss some of the design choices this protocol is based upon.

Why shall Bob add TX_B after TX_A has been validated? If TX_B is added to the chain before TX_A is issued, Alice could immediately claim $\$B$ from TX_B by revealing x_0 and then abort. In this way, Alice never learns x_1, however, she managed to steal $\$B$ from Bob.

Why do both parties need to sign to release the money in TX_A and TX_B? Clearly, the money in TX_A cannot be released with just a signature from Bob. Otherwise, Bob could receive $\$B$ from TX_A without ever revealing his secret x_1. Similarly, the money in TX_A cannot be released with just a signature from Alice, otherwise, as soon as TX_B appears on the chain, Alice claims both the money in TX_B (by revealing x_0) and that in TX_A (by signing a transaction with her key).

As for TX_B the issue is similar, but a bit trickier to see. If a single signature from Alice was sufficient, Alice could redeem the money in TX_B without ever revealing x_0. On the other hand, if a single signature from Bob was sufficient, Bob could wait for Alice to broadcast to the miners the transaction in which she claims the money in TX_B by revealing x_0. Then, he rushes to issue a competing transaction that moves the money in TX_B to his account (a single signature would suffice). Assuming Bob has a better position in the network, it is likely that his transaction gets validated before Alice's. At that point, Bob can even use x_0 and x_1 to claim the money in TX_A.

How to choose t_A and t_B? It is important that t_A is far enough into the future to allow the validation of TX_A, TX_B, the transaction that allows Alice to redeem the money in TX_B, and the transaction that allows Bob to redeem the money in TX_A. Similarly, t_B should be far enough to allow the validation of TX_A, TX_B and the transaction that allows Alice to redeem the money in TX_B. Notice however that t_B is too close to t_A, Alice can perform the following attack: instead of claiming the money in TX_B immediately, she could wait for the last second before before t_B. At that point, she could rush to issue TX'_A to claim the money in TX_A for herself. If Bob is not fast enough in claiming the money in TX_A before her, he may end up losing $\$B$.

A generalisation to N parties. The blueprint for game-theoretic fair swap can be generalised to the N party setting. Specifically, we now deal with N parties $P_1, \ldots, P_N$, where P_i holds the private input x_i. The goal is for everybody to reveal their private input in a fair way: if any malicious participant decides not to reveal their secret after seeing the secret of any other party, their behaviour should be penalised.

Once again, we assume that everyone holds a hash y_i of x_i for every $i \in [N]$. In this way, everybody can check that the revealed secrets are indeed what they were supposed to receive. Each party P_i for $i \in \{2, \ldots, N\}$ prepares two transactions:

$\mathsf{TX_{i-1}}$: This transaction moves $\$(i-1)B$ from P_i's account to a smart contract. The money is sent to P_{i-1}'s account if they provide a tuple $(x_1, \ldots, x_{i-1})$ such that $y_j = \mathsf{Hash}(x_j)$ for $j = 1, \ldots, i-1$. Alternatively, P_i can redeem the money by providing a transaction signed by all parties having input $\mathsf{TX_{i-1}}$.

$\mathsf{TX'_{i-1}}$: This is a time-lock transaction that sends the money of $\mathsf{TX_{i-1}}$ back to P_i's account. The time parameter is set to a time t_{i-1}.

On the other hand, P_1 prepares the following transactions:

$\mathsf{TX_N}$: This is a multi-input transaction that moves $\$B$ from each P_i's account to a smart contract. The money in the contract is sent to P_N's account if they provide a tuple $(x_1, \ldots, x_N)$ such that $y_j = \mathsf{Hash}(x_j)$ for $j = 1, \ldots, N$. Alternatively, the money can be redeemed by providing any transaction signed by all parties having input $\mathsf{TX_N}$.

$\mathsf{TX'_N}$: This is a multi-output, time-lock transaction that sends the money of $\mathsf{TX_N}$ back to the participants accounts. In particular, each P_j receives the same amount $\$B$. The time parameter is set to a time t_N.

The protocol proceeds as follows:

1. Each party P_i broadcasts the transaction $\mathsf{TX'_{i-1}}$ to everybody else.
2. Each party P_i broadcasts sends a signature on $\mathsf{TX'_{j-1}}$ and $\mathsf{TX'_N}$ to every other party P_j.
3. Party P_1 sends $\mathsf{TX_N}$ to everybody else. Everybody signs the transaction to add it to the chain.
4. For $i = N, \ldots, 2$, when $\mathsf{TX_i}$ has been successfully added to the chain, party P_N signs $\mathsf{TX_{i-1}}$ and broadcasts it to the miners to add it to the chain.
5. When $\mathsf{TX_1}$ has been successfully added to the chain, P_1 redeems the money in $\mathsf{TX_1}$ by revealing their secret x_1.
6. For $i = 2, \ldots, N-1$, when party P_{i-1} has redeemed the money in $\mathsf{TX_{i-1}}$ by revealing the tuple $(x_1, \ldots, x_{i-1})$, party P_i redeems the money in $\mathsf{TX_i}$ by revealing $(x_1, \ldots, x_i)$.
7. If nobody redeems the money in any TX_i by time t_i, the parties broadcast $\mathsf{TX'_i}$ to the miners to add it to the chain.

The result is the same as before: if nobody aborts, everybody gets to learn the tuple $(x_1, \ldots, x_N)$ without gaining or losing any money. On the other hand, if party P_i aborts after P_{i-1} has revealed $(x_1, \ldots, x_{i-1})$, they lose $\$(i-1)B$, each among $P_1, \ldots, P_{i-1}$ gains $\$B$.

2 Post-Quantum Security

Post-quantum cryptography studies classical constructions (i.e. protocols and algorithms that can run on normal machines) that retain their security properties even against quantum attackers. This is a particularly important challenge, as a good portion of cryptography at the base of our digital infrastructures is totally broken by quantum computers. This includes popular public-key encryption schemes such as RSA and Diffie-Hellman and digital signatures such as ECDSA and EdDSA.

For the moment, the threat is mainly theoretical. In the moment I am writing these notes, the world has not managed to build any quantum computer that allows computations at a scale large enough to even be close to running any of the algorithms that would break modern-day cryptography. Some even conjecture that quantum computations at such scale are simply impossible due to not-yet explored laws of physics (for instance, it could be that it is impossible to prevent large quantum states from collapsing or getting entangled with the environment).

Despite this, the job of a cryptographer requires us to be cautious. Even if quantum computers do not seem within sight, research in this area is making fast progress. Moreover, research in general tends to proceed at a bumpy pace, alternating between periods of calm and sudden bursts of improvement. In conclusion, due to the unpredictability of the future, it is always better that we take this threat seriously. This is further motivated by the fact that, in some applications, we not only need protections from attackers that live at the same time as us, we might also need protection from attackers in the future. A typical example is *harvest-now-decrypt-later*: an attacker could store today's digital communications, waiting for the development of quantum computers that allow them to decrypt it all. Similar threats exist for digital signatures: if large scale quantum computers become a reality, retrieving today's digital signature keys would become suddenly easy. An attacker could therefore try to rewrite history by crafting communications that never really happened.

2.1 How does a quantum computer work?

Of course, this course cannot delve into the details of how a quantum computer works. However, we will try to provide an intuition. The main concept we need is *superposition*.

Schrödinger's cat. You have probably heard of this experiment, devised by Erwin Schrödinger in a discussion with Albert Einstein.

Suppose that we have a box that is isolated from the external environment[3]. Inside the box, we have a cat, a flask of deadly poison, and some radioactive material. The flask is connected to a device which, when it detects the radiation produced by the decay of an atom (for instance, using a Geiger counter), releases the poison. The poor cat dies.

Now, radioactive decay is a quantum phenomenon. This means that, if the box isolates the inside of the box from the outside, we not only wouldn't know whether the cat is dead or alive, but we would be asking a completely meaningless question! From our perspective, the cat is neither dead or alive, it is instead in a *superposition* of both states, similarly to how the radioactive material is in a superposition of decayed and not-decayed. It is as if our reality refuses to commit to a story. For the moment, it just waits. When we open the box, the inside reality of the cat gets suddenly entangled with our reality. Consequently, our world experience picks one of the many possible paths according to some distribution: we will either see a dead cat or one that meows back at us.

It is easy to imagine that this is as if reality plays dice with us, deciding whether the cat is dead or alive based on the result of the roll. What happens is, however, more complex (in all senses): describing the state of the cat using a probability distribution would only tell us half of the story. To really understand what is happening, we would need to rely on a generalisation of probability distributions, one that does not represent the probability of an event using a real number in $[0, 1]$, but with a complex number of norm at most 1. Now, this is the last time we mention complex numbers in this course, let's go back to our quantum computers.

Computing on superposition. A quantum computer works a bit like the box in Schrödinger experiment: instead of storing a state as a deterministic bit string, it stores a state that is a superposition of bit strings. Instead of a single n-bit string, we store a superposition of 2^n strings. The new feature is that we can also compute on these superpositions without opening the box: by evaluating a circuit C on the superposition of the inputs, we obtain a superposition of the outputs. In some sense, we can compute 2^n evaluations of C (one for every string in superposition), at the cost of a single evaluation.

This sounds like an exponential speed-up. There is, however, a big caveat: how do we extract the result from the box? The world inside the box is isolated from ours, that means that no information can travel in and out. To get any information about the outcome of the computation, *we need to open the box*! However,

[3]This is not just a paper box, but a special box that isolates the inside at a quantum level. No mechanical, electromagnetic or quantum interaction occurs between inside and outside. In some sense, the challenges we encounter in building a quantum computer are the same as the challenges we encounter when we try to build the box of Schrödinger's experiment.

as soon as we do that, the inner world gets entangled with our world, reality suddenly decides to commit to a course, and from the 2^n possible executions, we end up with a single one chosen accordingly to a (generalised) probability distribution. All other realities that were in superposition with the one we see after opening the box are lost forever. They have become part of inaccessible parallel worlds.

To better understand this concept, consider again Schrödinger's experiment. If we see a lively cat when we open the box, we have no clue about the death it experienced in the parallel reality that was in superposition. We do not know if the cat had a quick or slow death, we do not know if it suffered or if it just fell asleep, that reality is not *our reality* and so it left no traces in the world we experience. Similarly, when we open the register of the quantum computer and we see the result of the computation, we loose all information about all the other realities that did not manifest. We do not know anything about the results of all the other evaluations, it is as if these never happened. We can try to open and close the box as we want, but the outcome of the computation never changes. The realities that were in superposition are lost forever, we cannot undo it.

In jargon, the process of opening the box is called *measurement*. Given that this operation unavoidably erases any trace of the realities that were in superposition, we say that the measurement makes the *state collapse*.

2.2 Are Quantum Computers More Powerful than Classical Computers?

The answer to this question is (likely) yes (but just in theory, given that we do not know whether quantum computations at scale are possible). It turns out that a quantum computer can perform all operations that a classical computer can do. There are, however, tasks where quantum computers (theoretically) offer levels of efficiency that cannot be achieved (sometimes, we just conjecture) on a classical computer. There are three main problems where quantum algorithms seem to have the upper hand over classical solutions: *search*, *period finding* and *quantum simulation*. The first problem is to find, given a function F and a database with N elements, an entry x such that $F(x) = 1$. The second problem is to find the period of a function $F : \mathbb{Z} \to \mathbb{R}$, i.e., we are looking for the minimal $T \in \mathbb{N} \setminus \{0\}$ such that $F(x) = F(x+T)$ for every $x \in \mathbb{Z}$. The third problem is to simulate quantum systems to analyse their properties. For instance, this could be used to learn the structure of complicated molecules (such as proteins). In cryptography, we are interested mainly in the first two problems, the third one, although cool and important, is more interesting for physicists, chemists and biologists.

Grover's search Algorithm. Imagine that we have a database with N entries and we want to find an element x such that $F(x) = 1$. If we did this computation on a classical machine, on average, we would to perform $O(N)$ operations until we find x. If x does not even exist, we would need to check all N entries until we are sure that x does not exist. It would seem that doing better than this is impossible, however, a quantum computer can!

The algorithm that allows for this is called Grover's search and allows us to recover x (or determine that x does not exist) using $O(\sqrt{N})$ (quantum) operations. Although quite exciting, this result has some negative consequences for cryptography (which bases its security on the hardness of performing certain operations).

For example, suppose that we hold a random n-bit key k. This could be the secret key for a symmetric encryption scheme, or for a public-key encryption scheme, or for a signature scheme. Let us assume that a quantum attacker has some way to verify whether a candidate key is actually k or not[4]. Then, using Grover's algorithm, they could find k in time $O(\sqrt{2^n}) = O(2^{n/2})$. This is still a computation that grows exponentially in n, so it is still quite expensive. However, it is a much better than the time $O(2^n)$ that a classical algorithm would require. To summarise, if we want the level of security against quantum attackers to be same as the level of security we currently have against classical attackers, *we need to double the length of all our keys*.

[4]This is typically the case for public-key encryption and signature schemes, where we can always test whether the candidate key allows us to decrypt/sign. For symmetric encryption schemes, the situation is a bit more complicated, but even in this case, this is usually not a problem assuming we have some ciphertexts on which we can try k.

Something similar applies to hash functions. If a hash function produces $2n$-bit digests, a classical algorithm would require on average $O(2^n)$ operations to find a collision. This is because, by the birthday bound, there are roughly $N^2/2^{2n}$ collisions in a set of N random elements, so all we need to do is to pick a random database of $N = O(2^n)$ elements and evaluate the hash function on all entries until we find a collision. Using Grover's search algorithm, a quantum computer can perform the same computation using $O(2^{n/2})$ (quantum) operations. In conclusion, if we want the level of security against quantum attackers to be same as the level of security we currently have against classical attackers, *we need to double the length of the output of all hash functions.*

Finally, there are consequences for PoW-based blockchains too. If we use a hash function with n-bit output and the hardness parameter is set to T, a classical computer needs on average $O(2^n/T)$ evaluations of the hash function until it manages to mine the next block. Using Grover's search, a quantum computer would require, however, $O(\sqrt{2^n/T})$ (quantum) operations. In other words, in a world in which miners are all quantum machines, to maintain the hardness of mining the same (and therefore, the rate at which the blocks are published), we would need to increase the value of T or the output size of the hash function (or both).

Quantum Fourier Transform. While Grover's algorithm gives a significant but not game-changing advantage to a quantum attacker, the next algorithm we are going to discuss has totally destructive consequences for some cryptographic constructions. These include RSA, Diffie-Hellman, and ECDSA and EdDSA signatures.

Suppose that we hold the description of a function $F : \mathbb{Z} \to \mathbb{R}$ which is periodic, i.e., there exist a non-zero element $T \in \mathbb{N}$ such that $F(x) = F(x + T)$ for every $x \in \mathbb{Z}$. Suppose also that we know an upper bound N on T. The *Quantum Fourier Transform* algorithm (QFT) allows us to find T in time $\mathsf{polylog}(N)$. Classical algorithms, on the other hand, try to find T by probing F at multiple points, hoping to see a repeating pattern. Even the best known solutions of this kind, which choose the points in clever ways, require $N^{\Omega(1)}$ operations. This is really an exponential speed-up!

To fully understand the degree to which QFT is problematic for computer security, we need to consider that cryptographic tools are typically based on the (conjectured) hardness of few well-studied problems. For instance, RSA bases its security on the hardness of factoring large integers. Diffie-Hellman, ECDSA and EdDSA are instead based on the hardness of variants of a problem called *discrete logarithm*[5].

The bad news is that factoring and discrete logarithm can both be rephrased as period finding problems. Consequently, unlike a classical attacker, a quantum computer could solve these problems *efficiently*. This completely breaks the (post-quantum) security of all cryptographic tools that are based on them.

2.3 Quantumly Hard Problems

If factoring and discrete logarithm are no longer hard problems in the quantum world, on which problems can we base the security of post-quantum constructions? One option would be to base it on current tools that are not broken by quantum computers, for instance, hash functions and symmetric encryption schemes such as AES. This could work for primitives like signature schemes (via a framework called MPC-in-the-head) and succinct, zero-knowledge arguments. The question here is whether we can achieve the same level of efficiency of the old constructions. There is, however, no known way to build public-key encryption from hash functions and symmetric encryption (the general conjecture is that this is impossible). That is why we need to rely on new assumptions. We list some of the most popular ones.

Lattices: Lattices are a geometric structure over the n-dimentional space $\mathbb{R}^n$, consisting of points arranged on a regular grid. Typical examples of lattice-based problems that are conjectured to be hard to solve by quantum computers are Learning with Errors (LWE), which essentially asks us to distinguish between random points of $\mathbb{R}^n$ and random points that are close to the lattice. Another example is

[5]The basic form of the discrete logarithm problem is the following: given a prime p, and values g and $g^x \bmod p$, the goal is to determine x. There exists a variant of the problem over an algebraic structure called elliptic curves, this is what is used for instance in ECDSA and EdDSA.

the Shortest Interger Solution (SIS), which asks us to find a non-zero lattice point that is close to the origin. A final example is NTRU.

Error Correction Codes: The most typical example of quantumly hard problem in the code setting is Learning Parity with Noise (LPN). The problem here is usually to distinguish between random strings of bits, and random strings of bits that are close to a valid codewords (distance is defined using the Hamming distance).

Multivariate Quadratic Equations: These problems typically ask to find a solution to systems of multivariate equations of degree 2.

Isogenies: These problems are based on complex algebraic-geometric structures called isogenies. An isogeny consists of a homomorphic map between algebraic structures called elliptic curves. The main problem here is, given two curves, to find the isogeny that maps the first one to the second one (don't worry, I won't ask this at the exam).

2.4 Summary

Let's summarise how quantum computing at scale would impact cryptographic tools:

Symmetric Encryption: Most constructions, including popular ones like AES, are conjectured to remain secure. However, we would need to increase the key size to counteract the quantum speed-up offered by Grover's search.

Hash Functions: Same as symmetric encryption. Most constructions, including popular ones like SHA2 and SHA3, are conjectured to remain secure. However, we would need to increase the digest size to counteract the quantum speed-up offered by Grover's search.

Public-key Encryption: QFT would completely break the security of popular constructions such as RSA, Diffie-Hellman (both over finite fields and elliptic curves). More in general, all constructions that are based on the hardness of factoring (e.g. Paillier cryptosystem) or discrete logarithm (e.g. ElGamal) are also completely broken. This includes also pairing-based constructions (pairing-based cryptography relies on the hardness of a fancier version of discrete logarithm). Public-key encryption schemes need to be substituted by any of the post-quantum alternatives we describe below.

Signature Schemes: QFT would completely break the security of popular constructions such as RSA, DSA, ECDSA and EdDSA. More in general, all constructions that are based on the hardness of factoring or discrete logarithm are also completely broken. This includes also pairing-based constructions (e.g. BLS signatures), which typically are used in fancier signature constructions like those we saw in Lecture 8 to achieve better anonymity. Signature schemes need to be substituted by any of the post-quantum alternatives we describe below.

Zero-Knowledge and Succinct Arguments: Many constructions base their security on the hardness of factoring, discrete logarithm or on pairing techniques. All the constructions of this type are insecure against quantum attackers and need to be substituted with valid post-quantum alternative (for instance, lattice-based ones).

So, how do we fix cryptography? What post-quantum options do we have? Research in this area is still ongoing, however, we already have several post-quantum alternatives for public-key encryption, signatures, and zero-knowledge and succinct arguments. These achieve comparable levels of efficiency to the quantumly broken schemes that are currently in use. A few years ago, the National Institute of Standards and Technology (NIST) has initiated a competition to select the new post-quantum tools that will secure the Internet of the future. The competition is still ongoing, however, some schemes have already been selected. RSA and Diffie-Hellman will be substituted by Kyber (renamed ML-KEM), a lattice-based key encapsulation mechanism.

On the other hand, RSA signatures, ECDSA and EdDSA will be substituted by Dilithium (renamed ML-DSA), FALCON (renamed FN-DSA), and SPHINCS+ (renamed SLH-DSA). The first two are based on different variants of lattice-based problems, the third one is based on hash functions. There are, however, many other candidates that are currently under consideration.